

Kelp1 - Export Seluruh Kode

Isi: code cells notebook Kelp1_EDA.ipynb dan file Streamlit Kelp1_app.py.

Dibuat otomatis untuk output proyek UTS NLP ABSA Kelompok 1.

Generated: 2026-04-28 08:30:37

Catatan: PDF ini berisi kode, bukan output visual.

1. Notebook: notebook/Kelp1_EDA.ipynb

Code Cell 1

```

from pathlib import Path
import json
import re
from collections import Counter, defaultdict

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from IPython.display import display

pd.set_option("display.max_colwidth", 120)

DATA_DIR = Path("../dataset")
ANNOTATION_PATH = DATA_DIR / "Kelp1_dataset_anotasi.jsonl"
AGGREGATED_PATH = DATA_DIR / "Kelp1_dataset_anotasi_aggregated.jsonl"
RAW_CSV_PATH = DATA_DIR / "Kelp1_dataset_1.csv"
CLEAN_CSV_PATH = DATA_DIR / "Kelp1_dataset_2.csv"
IRR_NER_PATH = DATA_DIR / "Kelp1_irr_ner.json"
IRR_TEXTCAT_PATH = DATA_DIR / "Kelp1_irr_textcat.json"

LABELS = [
    "PRODUCT_POSITIVE", "PRODUCT_NEGATIVE", "PRODUCT_NEUTRAL",
    "PRICE_POSITIVE", "PRICE_NEGATIVE", "PRICE_NEUTRAL",
    "PLACE_POSITIVE", "PLACE_NEGATIVE", "PLACE_NEUTRAL",
    "PROMOTION_POSITIVE", "PROMOTION_NEGATIVE", "PROMOTION_NEUTRAL",
    "OUT_OF_TOPIC",
]

def load_jsonl(path):
    rows = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            if line.strip():
                rows.append(json.loads(line))
    return rows

def clean_text(text):
    text = str(text)
    text = re.sub(r"\s+", " ", text).strip()
    text = text.replace("\u200b", "")
    return text

def word_count(text):
    return len(re.findall(r"\b\w+\b", str(text).lower()))

def parse_label(label):
    if label == "OUT_OF_TOPIC":
        return "OUT_OF_TOPIC", "OUT_OF_TOPIC"
    parts = label.rsplit("_", 1)
    return parts[0], parts[1] if len(parts) == 2 else ""

```

Code Cell 2

```

annotation_rows = load_jsonl(ANNOTATION_PATH)

print("Jumlah baris anotasi:", len(annotation_rows))
print("Jumlah review unik berdasarkan _input_hash:", len({row["_input_hash"] for row in annotation_rows}))
print("Jumlah review unik berdasarkan text:", len({row["text"] for row in annotation_rows}))
print("Jumlah annotator:", len({row.get("_annotator_id", "") for row in annotation_rows}))

pd.DataFrame({
    "annotator_id": list(Counter(row.get("_annotator_id", "") for row in annotation_rows).keys()),
    "jumlah_anotasi": list(Counter(row.get("_annotator_id", "") for row in annotation_rows).values())
})

```

Code Cell 3

```

groups = defaultdict(list)
for row in annotation_rows:
    groups[row["_input_hash"]].append(row)

aggregated = []
for idx, (input_hash, items) in enumerate(groups.items(), start=1):
    text = items[0]["text"]
    label_votes = Counter()
    for item in items:
        for label in set(item.get("accept", [])):
            label_votes[label] += 1

majority_labels = [label for label in LABELS if label_votes.get(label, 0) >= 2]
union_labels = [label for label in LABELS if label_votes.get(label, 0) > 0]

```

```

spans_all = []
for item in items:
    annotator_id = item.get("_annotator_id", "")
    for span in item.get("spans", []):
        start, end = span.get("start"), span.get("end")
        spans_all.append({
            "start": start,
            "end": end,
            "token_start": span.get("token_start"),
            "token_end": span.get("token_end"),
            "label": span.get("label"),
            "entity_text": text[start:end] if isinstance(start, int) and isinstance(end, int) else "",
            "annotator_id": annotator_id,
        })

cleaned = clean_text(text)
aggregated.append({
    "review_id": f"Kelp1_{idx:04d}",
    "input_hash": input_hash,
    "category": "Kuliner",
    "business_name": "",
    "rating": "",
    "review_text": text,
    "clean_review_text": cleaned,
    "word_count": word_count(cleaned),
    "token_count": len(items[0].get("tokens", [])),
    "n_annotators": len(items),
    "annotators": sorted({item.get("_annotator_id", "") for item in items}),
    "majority_labels": majority_labels,
    "union_labels": union_labels,
    "label_votes": dict(label_votes),
    "spans_all": spans_all,
    "spans_majority": [span for span in spans_all if span.get("label") in majority_labels],
})

df_reviews = pd.DataFrame([
    {"review_id": item["review_id"],
    "category": item["category"],
    "business_name": item["business_name"],
    "rating": item["rating"],
    "review_text": item["review_text"],
    "clean_review_text": item["clean_review_text"],
    "word_count": item["word_count"],
    "token_count": item["token_count"],
    "n_annotators": item["n_annotators"],
    "majority_labels": ", ".join(item["majority_labels"]),
    "union_labels": ", ".join(item["union_labels"]),
    } for item in aggregated])

df_labels = []
df_entities = []
for item in aggregated:
    for label in LABELS:
        aspect, sentiment = parse_label(label)
        df_labels.append({
            "review_id": item["review_id"],
            "label": label,
            "aspect": aspect,
            "sentiment": sentiment,
            "vote_count": int(item["label_votes"].get(label, 0)),
            "is_majority": label in item["majority_labels"],
            "is_union": label in item["union_labels"],
        })
    for span in item["spans_all"]:
        aspect, sentiment = parse_label(span.get("label", ""))
        df_entities.append({
            "review_id": item["review_id"],
            "annotator_id": span.get("annotator_id", ""),
            "entity_text": span.get("entity_text", ""),
            "label": span.get("label", ""),
            "aspect": aspect,
            "sentiment": sentiment,
            "start": span.get("start"),
            "end": span.get("end"),
            "is_majority_label": span.get("label") in item["majority_labels"],
        })

df_labels = pd.DataFrame(df_labels)
df_entities = pd.DataFrame(df_entities)

print("Review unik:", len(df_reviews))
print("Tabel label:", df_labels.shape)

```

```
print("Tabel entitas:", df_entities.shape)
display(df_reviews.head())
```

Code Cell 4

```
df_dataset_1 = df_reviews[["review_id", "category", "business_name", "review_text", "rating"]].copy()
df_dataset_2 = df_reviews[["review_id", "category", "business_name", "clean_review_text", "rating", "word_count",
    "token_count"]].copy()
df_dataset_2 = df_dataset_2.rename(columns={"clean_review_text": "review_text"})
df_dataset_2 = df_dataset_2.drop_duplicates(subset=["review_text"]).reset_index(drop=True)

df_dataset_1.to_csv(RAW_CSV_PATH, index=False, encoding="utf-8-sig")
df_dataset_2.to_csv(CLEAN_CSV_PATH, index=False, encoding="utf-8-sig")

print("Dataset 1 tersimpan:", RAW_CSV_PATH, "| baris:", len(df_dataset_1))
print("Dataset 2 tersimpan:", CLEAN_CSV_PATH, "| baris:", len(df_dataset_2))
display(df_dataset_2.head())
```

Code Cell 5

```
label_distribution = (
    df_labels[df_labels["is_majority"]]
    .groupby(["label", "aspect", "sentiment"])
    .size()
    .reset_index(name="jumlah_review")
    .sort_values("jumlah_review", ascending=False)
)

display(label_distribution)

fig, ax = plt.subplots(figsize=(10, 6))
plot_data = label_distribution.sort_values("jumlah_review")
ax.barh(plot_data["label"], plot_data["jumlah_review"])
ax.set_title("Distribusi Label ABSA (Mayoritas >= 2 dari 3 Annotator)")
ax.set_xlabel("Jumlah review")
ax.set_ylabel("Label")
fig.tight_layout()
plt.show()
```

Code Cell 6

```
entity_distribution = (
    df_entities[df_entities["is_majority_label"]]
    .groupby(["label", "aspect", "sentiment"])
    .size()
    .reset_index(name="jumlah_entitas")
    .sort_values("jumlah_entitas", ascending=False)
)

display(entity_distribution)

fig, ax = plt.subplots(figsize=(10, 6))
plot_data = entity_distribution.sort_values("jumlah_entitas")
ax.barh(plot_data["label"], plot_data["jumlah_entitas"])
ax.set_title("Distribusi Entitas ABSA/NER pada Label Mayoritas")
ax.set_xlabel("Jumlah entitas")
ax.set_ylabel("Label")
fig.tight_layout()
plt.show()
```

Code Cell 7

```
display(df_reviews[["word_count", "token_count"]].describe())

fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(df_reviews["word_count"], bins=30)
ax.set_title("Distribusi Panjang Review Berdasarkan Jumlah Kata")
ax.set_xlabel("Jumlah kata")
ax.set_ylabel("Jumlah review")
fig.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(8, 4))
ax.boxplot(df_reviews["word_count"], vert=False)
ax.set_title("Boxplot Panjang Review")
ax.set_xlabel("Jumlah kata")
fig.tight_layout()
plt.show()
```

Code Cell 8

```
aspect_distribution = (
    df_labels[df_labels["is_majority"]]
    .groupby("aspect")
    .size()
    .reset_index(name="jumlah_review")
)
```

```

        .sort_values("jumlah_review", ascending=False)
    )

sentiment_distribution = (
    df_labels[df_labels["is_majority"]]
    .groupby("sentiment")
    .size()
    .reset_index(name="jumlah_review")
    .sort_values("jumlah_review", ascending=False)
)

display(aspect_distribution)
display(sentiment_distribution)

fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(aspect_distribution["aspect"], aspect_distribution["jumlah_review"])
ax.set_title("Distribusi Data per Aspek")
ax.set_xlabel("Aspek")
ax.set_ylabel("Jumlah review")
ax.tick_params(axis="x", rotation=30)
fig.tight_layout()
plt.show()

```

Code Cell 9

```

label_matrix = pd.DataFrame(0, index=df_reviews["review_id"], columns=LABELS)

for _, row in df_labels[df_labels["is_majority"]].iterrows():
    label_matrix.loc[row["review_id"], row["label"]] = 1

correlation_matrix = label_matrix.corr().fillna(0)
display(correlation_matrix.round(2))

fig, ax = plt.subplots(figsize=(10, 8))
im = ax.imshow(correlation_matrix.values, vmin=-1, vmax=1)
ax.set_xticks(range(len(LABELS)))
ax.set_yticks(range(len(LABELS)))
ax.set_xticklabels(LABELS, rotation=90, fontsize=8)
ax.set_yticklabels(LABELS, fontsize=8)
ax.set_title("Matriks Korelasi Antar Label ABSA")
fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
fig.tight_layout()
plt.show()

```

Code Cell 10

```

if IRR_TEXTCAT_PATH.exists():
    irr_textcat = json.load(open(IRR_TEXTCAT_PATH, encoding="utf-8"))
    irr_textcat_df = pd.DataFrame.from_dict(irr_textcat, orient="index").reset_index(names="label")
    display(irr_textcat_df[["label", "n_examples", "n_annotators", "percent_agreement", "kripp_alpha", "gwet_ac2"]])
else:
    print("File IRR textcat tidak ditemukan.")

if IRR_NER_PATH.exists():
    irr_ner = json.load(open(IRR_NER_PATH, encoding="utf-8"))
    irr_summary = pd.DataFrame([
        {"metric": key, "value": irr_ner.get(key)}
        for key in ["n_examples", "n_categories", "n_coincident_examples", "n_annotators", "avg_raters_per_example",
                    "pairwise_f1", "pairwise_recall", "pairwise_precision"]
        if key in irr_ner
    ])
    display(irr_summary)
else:
    print("File IRR NER tidak ditemukan.")

```

2. Streamlit: streamlit/Kelp1_app.py

```

import json
import re
from collections import Counter, defaultdict
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import streamlit as st

LABELS = [
    "PRODUCT_POSITIVE", "PRODUCT_NEGATIVE", "PRODUCT_NEUTRAL",
    "PRICE_POSITIVE", "PRICE_NEGATIVE", "PRICE_NEUTRAL",
    "PLACE_POSITIVE", "PLACE_NEGATIVE", "PLACE_NEUTRAL",
    "PROMOTION_POSITIVE", "PROMOTION_NEGATIVE", "PROMOTION_NEUTRAL",
    "OUT_OF_TOPIC",
]

def get_paths():
    app_dir = Path(__file__).resolve().parent
    project_dir = app_dir.parent
    data_dir = project_dir / "dataset"
    if not data_dir.exists():
        data_dir = Path.cwd() / "dataset"
    return {
        "project_dir": project_dir,
        "data_dir": data_dir,
        "annotation": data_dir / "Kelp1_dataset_anotasi.jsonl",
        "aggregated": data_dir / "Kelp1_dataset_anotasi_aggregated.jsonl",
        "raw_csv": data_dir / "Kelp1_dataset_1.csv",
        "clean_csv": data_dir / "Kelp1_dataset_2.csv",
        "irr_ner": data_dir / "Kelp1_irr_ner.json",
        "irr_textcat": data_dir / "Kelp1_irr_textcat.json",
    }

def clean_text(text):
    text = str(text)
    text = re.sub(r"\s+", " ", text).strip()
    text = text.replace("\u200b", "")
    return text

def word_count(text):
    return len(re.findall(r"\b\w+\b", str(text).lower()))

def parse_label(label):
    if label == "OUT_OF_TOPIC":
        return "OUT_OF_TOPIC", "OUT_OF_TOPIC"
    parts = label.rsplit("_", 1)
    if len(parts) == 2:
        return parts[0], parts[1]
    return label, ""

def load_jsonl(path):
    rows = []
    if not Path(path).exists():
        return rows
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            if line.strip():
                rows.append(json.loads(line))
    return rows

def load_json(path):
    if not Path(path).exists():
        return None
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)

@st.cache_data(show_spinner=False)
def aggregate_from_annotation(annotation_path):
    rows = load_jsonl(annotation_path)
    groups = defaultdict(list)
    for row in rows:

```

```

input_hash = row.get("_input_hash", row.get("text", ""))
groups[input_hash].append(row)

aggregated = []
for idx, (input_hash, items) in enumerate(groups.items(), start=1):
    text = items[0].get("text", "")
    votes = Counter()
    for item in items:
        for label in set(item.get("accept", [])):
            votes[label] += 1

    majority_labels = [label for label in LABELS if votes.get(label, 0) >= 2]
    union_labels = [label for label in LABELS if votes.get(label, 0) > 0]
    spans_all = []
    for item in items:
        annotator_id = item.get("_annotator_id", "")
        for span in item.get("spans", []):
            start = span.get("start")
            end = span.get("end")
            spans_all.append({
                "start": start,
                "end": end,
                "token_start": span.get("token_start"),
                "token_end": span.get("token_end"),
                "label": span.get("label"),
                "entity_text": text[start:end] if isinstance(start, int) and isinstance(end, int) else "",
                "annotator_id": annotator_id,
            })

    aggregated.append({
        "review_id": f"Kelp1_{idx:04d}",
        "input_hash": input_hash,
        "category": "Kuliner",
        "business_name": "",
        "rating": "",
        "review_text": text,
        "clean_review_text": clean_text(text),
        "word_count": word_count(text),
        "token_count": len(items[0].get("tokens", [])),
        "n_annotators": len(items),
        "annotators": sorted({item.get("_annotator_id", "") for item in items}),
        "majority_labels": majority_labels,
        "union_labels": union_labels,
        "label_votes": dict(votes),
        "spans_all": spans_all,
        "spans_majority": [span for span in spans_all if span.get("label") in majority_labels],
    })
return aggregated, len(rows)

@st.cache_data(show_spinner=False)
def load_data():
    paths = get_paths()
    if paths["aggregated"].exists():
        aggregated = load_jsonl(paths["aggregated"])
        annotation_rows = len(load_jsonl(paths["annotation"]))
    else:
        aggregated, annotation_rows = aggregate_from_annotation(str(paths["annotation"]))

    review_records = []
    label_records = []
    entity_records = []

    for item in aggregated:
        majority_labels = item.get("majority_labels", [])
        union_labels = item.get("union_labels", [])
        label_votes = item.get("label_votes", {})

        review_records.append({
            "review_id": item.get("review_id", ""),
            "input_hash": item.get("input_hash", ""),
            "category": item.get("category", "Kuliner") or "Kuliner",
            "business_name": item.get("business_name", ""),
            "rating": item.get("rating", ""),
            "review_text": item.get("review_text", ""),
            "clean_review_text": item.get("clean_review_text", clean_text(item.get("review_text", ""))),
            "word_count": item.get("word_count", word_count(item.get("review_text", ""))),
            "token_count": item.get("token_count", 0),
            "n_annotators": item.get("n_annotators", 0),
            "majority_labels": ", ".join(majority_labels),
            "union_labels": ", ".join(union_labels),
        })

```

```

for label in LABELS:
    aspect, sentiment = parse_label(label)
    vote_count = int(label_votes.get(label, 0))
    label_records.append({
        "review_id": item.get("review_id", ""),
        "label": label,
        "aspect": aspect,
        "sentiment": sentiment,
        "vote_count": vote_count,
        "is_majority": label in majority_labels,
        "is_union": label in union_labels,
    })

spans = item.get("spans_all", [])
for span in spans:
    label = span.get("label", "")
    aspect, sentiment = parse_label(label)
    entity_records.append({
        "review_id": item.get("review_id", ""),
        "annotator_id": span.get("annotator_id", ""),
        "label": label,
        "aspect": aspect,
        "sentiment": sentiment,
        "entity_text": span.get("entity_text", ""),
        "start": span.get("start", None),
        "end": span.get("end", None),
        "is_majority_label": label in majority_labels,
    })

reviews_df = pd.DataFrame(review_records)
labels_df = pd.DataFrame(label_records)
entities_df = pd.DataFrame(entity_records)
return reviews_df, labels_df, entities_df, annotation_rows

def filter_reviews(reviews_df, labels_df, selected_categories, selected_labels, query):
    filtered = reviews_df.copy()
    if selected_categories:
        filtered = filtered[filtered["category"].isin(selected_categories)]
    if query:
        q = query.lower().strip()
        filtered = filtered[
            filtered["review_text"].str.lower().str.contains(q, regex=False, na=False)
            | filtered["business_name"].astype(str).str.lower().str.contains(q, regex=False, na=False)
        ]
    if selected_labels:
        selected_ids = labels_df[
            (labels_df["label"].isin(selected_labels)) & (labels_df["is_majority"])
        ][["review_id"].unique()]
        filtered = filtered[filtered["review_id"].isin(selected_ids)]
    return filtered

def label_count_df(labels_df, only_majority=True):
    data = labels_df[labels_df["is_majority"]] if only_majority else labels_df[labels_df["is_union"]]
    counts = data.groupby(["label", "aspect", "sentiment"]).size().reset_index(name="jumlah_review")
    return counts.sort_values("jumlah_review", ascending=False)

def entity_count_df(entities_df):
    if entities_df.empty:
        return pd.DataFrame(columns=["label", "aspect", "sentiment", "jumlah_entitas"])
    data = entities_df[entities_df["is_majority_label"]].copy()
    counts = data.groupby(["label", "aspect", "sentiment"]).size().reset_index(name="jumlah_entitas")
    return counts.sort_values("jumlah_entitas", ascending=False)

def make_bar_chart(df, x_col, y_col, title, xlabel):
    fig, ax = plt.subplots(figsize=(9, max(4, 0.35 * len(df))))
    ax.barh(df[y_col], df[x_col])
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.invert_yaxis()
    fig.tight_layout()
    return fig

def make_length_histogram(reviews_df):
    fig, ax = plt.subplots(figsize=(9, 4.8))
    ax.hist(reviews_df["word_count"].dropna(), bins=30)
    ax.set_title("Distribusi Panjang Review")
    ax.set_xlabel("Jumlah kata")
    ax.set_ylabel("Jumlah review")

```

```

fig.tight_layout()
return fig

def make_correlation_matrix(reviews_df, labels_df):
    review_ids = reviews_df["review_id"].tolist()
    matrix = pd.DataFrame(0, index=review_ids, columns=LABELS)
    majority = labels_df[labels_df["is_majority"]][["review_id", "label"]]
    for _, row in majority.iterrows():
        if row["review_id"] in matrix.index and row["label"] in matrix.columns:
            matrix.loc[row["review_id"], row["label"]] = 1
    corr = matrix.corr().fillna(0)

    fig, ax = plt.subplots(figsize=(10, 8))
    im = ax.imshow(corr.values, vmin=-1, vmax=1)
    ax.set_xticks(range(len(corr.columns)))
    ax.set_yticks(range(len(corr.index)))
    ax.set_xticklabels(corr.columns, rotation=90, fontsize=8)
    ax.set_yticklabels(corr.index, fontsize=8)
    ax.set_title("Matriks Korelasi Antar Label")
    fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    fig.tight_layout()
    return fig, corr

def page_overview(reviews_df, labels_df, entities_df, annotation_rows):
    st.title("Kelp1 Dataset Explorer - ABSA Google Places")
    st.caption("Aplikasi eksplorasi dataset anotasi Aspect-Based Sentiment Analysis untuk Kelompok 1.")

    annotator_count = int(reviews_df["n_annotators"].max()) if not reviews_df.empty else 0
    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Review unik", f"{len(reviews_df):,}")
    col2.metric("Baris anotasi", f"{annotation_rows:,}")
    col3.metric("Annotator", f"{annotator_count:,}")
    col4.metric("Entitas NER", f"{len(entities_df):,}")

    st.info(
        "Kolom business_name dan rating disediakan di tabel, tetapi bernilai kosong karena metadata tersebut "
        "tidak tersedia pada file anotasi yang digunakan."
    )

    counts = label_count_df(labels_df)
    st.subheader("Distribusi Label ABSA")
    st.dataframe(counts, use_container_width=True, hide_index=True)
    if not counts.empty:
        plot_df = counts.rename(columns={"label": "Label", "jumlah_review": "Jumlah"})
        st.pyplot(make_bar_chart(plot_df, "Jumlah", "Label", "Distribusi Label Mayoritas", "Jumlah review"))

    st.subheader("Contoh Data Review")
    columns = ["review_id", "category", "business_name", "rating", "review_text", "majority_labels", "word_count"]
    st.dataframe(reviews_df[columns].head(20), use_container_width=True, hide_index=True)

def page_browse(reviews_df, labels_df, filtered_reviews):
    st.title("Browse Reviews")
    st.caption("Gunakan filter di sidebar untuk mencari review berdasarkan kategori, label aspek-sentimen, dan kata kunci.")

    st.write(f"Menampilkan **{len(filtered_reviews):,}** dari **{len(reviews_df):,}** review unik.")
    table_cols = ["review_id", "category", "business_name", "rating", "review_text", "majority_labels", "word_count"]
    st.dataframe(filtered_reviews[table_cols], use_container_width=True, hide_index=True, height=450)

    csv_bytes = filtered_reviews[table_cols].to_csv(index=False).encode("utf-8-sig")
    st.download_button("Download data terfilter (CSV)", data=csv_bytes, file_name="Kelp1_filtered_reviews.csv",
        mime="text/csv")

    st.subheader("Detail Review")
    chosen = st.selectbox("Pilih review_id", filtered_reviews["review_id"].tolist() if not filtered_reviews.empty else [])
    if chosen:
        row = filtered_reviews[filtered_reviews["review_id"] == chosen].iloc[0]
        st.markdown(f"***{row['review_id']}***")
        st.write(row["review_text"])
        st.write("Label mayoritas:", row["majority_labels"] if row["majority_labels"] else "-")
        st.write("Jumlah kata:", row["word_count"])

        label_view = labels_df[(labels_df["review_id"] == chosen) & (labels_df["is_union"])].copy()
        st.dataframe(label_view[["label", "aspect", "sentiment", "vote_count", "is_majority"]],
            use_container_width=True, hide_index=True)

def page_absa(labels_df, entities_df, filtered_reviews, selected_labels):
    st.title("Panel ABSA dan NER")

```

```

valid_ids = set(filtered_reviews["review_id"].tolist())
label_view = labels_df[labels_df["review_id"].isin(valid_ids)].copy()
entity_view = entities_df[entities_df["review_id"].isin(valid_ids)].copy()

if selected_labels:
    label_view = label_view[label_view["label"].isin(selected_labels)]
    entity_view = entity_view[entity_view["label"].isin(selected_labels)]

tab1, tab2, tab3 = st.tabs(["Label ABSA", "Entitas NER", "Ringkasan"])
with tab1:
    st.subheader("Tabel Label ABSA")
    st.write("Kolom `is_majority=True` berarti label dipilih minimal 2 dari 3 annotator.")
    label_table = label_view[label_view["is_union"] | label_view["is_majority"]].copy()
    st.dataframe(
        label_table[["review_id", "label", "aspect", "sentiment", "vote_count", "is_majority"]],
        use_container_width=True,
        hide_index=True,
        height=500,
    )
with tab2:
    st.subheader("Tabel Entitas NER")
    if entity_view.empty:
        st.warning("Tidak ada entitas NER untuk filter saat ini.")
    else:
        st.dataframe(
            entity_view[["review_id", "annotator_id", "entity_text", "label", "aspect", "sentiment", "start",
                        "end", "is_majority_label"]],
            use_container_width=True,
            hide_index=True,
            height=500,
        )
with tab3:
    st.subheader("Distribusi Label dan Entitas")
    label_summary = label_count_df(label_view)
    entity_summary = entity_count_df(entity_view)
    c1, c2 = st.columns(2)
    with c1:
        st.markdown("***Label ABSA mayoritas***")
        st.dataframe(label_summary, use_container_width=True, hide_index=True)
    with c2:
        st.markdown("***Entitas NER pada label mayoritas***")
        st.dataframe(entity_summary, use_container_width=True, hide_index=True)

def page_statistics(reviews_df, labels_df, entities_df, filtered_reviews):
    st.title("Statistics")
    st.caption("Ringkasan statistik minimal: jumlah data, distribusi panjang review, dan distribusi data per aspek.")

    st.subheader("Jumlah Data")
    c1, c2, c3 = st.columns(3)
    c1.metric("Review pada filter", f"{len(filtered_reviews):,}")
    c2.metric("Rata-rata kata", f"{filtered_reviews['word_count'].mean():.2f}" if not filtered_reviews.empty else "0")
    c3.metric("Median kata", f"{filtered_reviews['word_count'].median():.0f}" if not filtered_reviews.empty else "0")

    if not filtered_reviews.empty:
        st.pyplot(make_length_histogram(filtered_reviews))

    valid_ids = set(filtered_reviews["review_id"].tolist())
    label_view = labels_df[labels_df["review_id"].isin(valid_ids)]
    entity_view = entities_df[entities_df["review_id"].isin(valid_ids)]

    st.subheader("Distribusi Data per Aspek")
    aspect_summary = label_view[label_view["is_majority"]].groupby("aspect").size().reset_index(name="jumlah_review")
    st.dataframe(aspect_summary.sort_values("jumlah_review", ascending=False), use_container_width=True,
                 hide_index=True)
    if not aspect_summary.empty:
        fig, ax = plt.subplots(figsize=(7, 4))
        ax.bar(aspect_summary["aspect"], aspect_summary["jumlah_review"])
        ax.set_title("Distribusi Review per Aspek")
        ax.set_xlabel("Aspek")
        ax.set_ylabel("Jumlah review")
        ax.tick_params(axis="x", rotation=30)
        fig.tight_layout()
        st.pyplot(fig)

    st.subheader("Distribusi Entitas ABSA")
    ent_summary = entity_count_df(entity_view)
    st.dataframe(ent_summary, use_container_width=True, hide_index=True)
    if not ent_summary.empty:
        plot_df = ent_summary.rename(columns={"label": "Label", "jumlah_entitas": "Jumlah"})
        st.pyplot(make_bar_chart(plot_df, "Jumlah", "Label", "Distribusi Entitas NER", "Jumlah entitas"))

    st.subheader("Matriks Korelasi Antar Label")

```

```

if len(filtered_reviews) < 2:
    st.warning("Matriks korelasi membutuhkan minimal 2 review.")
else:
    fig, corr = make_correlation_matrix(filtered_reviews, label_view)
    st.pyplot(fig)
    with st.expander("Lihat nilai korelasi"):
        st.dataframe(corr, use_container_width=True)

def page_irr():
    st.title("IRR")
    st.caption("Menampilkan hasil IRR dari file yang sudah disiapkan dosen/kelompok.")

    paths = get_paths()
    irr_textcat = load_json(paths["irr_textcat"])
    irr_ner = load_json(paths["irr_ner"])

    st.subheader("IRR Text Classification / Multi-label")
    if irr_textcat:
        irr_df = pd.DataFrame.from_dict(irr_textcat, orient="index").reset_index(names="label")
        display_cols = ["label", "n_examples", "n_annotators", "percent_agreement", "kripp_alpha", "gwet_ac2"]
        available_cols = [col for col in display_cols if col in irr_df.columns]
        st.dataframe(irr_df[available_cols], use_container_width=True, hide_index=True)
    else:
        st.warning("File Kelp1_irr_textcat.json tidak ditemukan.")

    st.subheader("IRR NER / Span")
    if irr_ner:
        summary_keys = [
            "n_examples", "n_categories", "n_coincident_examples", "n_single_annotation",
            "n_annotators", "avg_raters_per_example", "pairwise_f1", "pairwise_recall", "pairwise_precision",
        ]
        summary_rows = [{"metric": key, "value": irr_ner.get(key)} for key in summary_keys if key in irr_ner]
        st.dataframe(pd.DataFrame(summary_rows), use_container_width=True, hide_index=True)

        metrics = irr_ner.get("metrics_per_label", {})
        if metrics:
            metrics_df = pd.DataFrame.from_dict(metrics, orient="index").reset_index(names="label")
            st.markdown("***Metrics per label***")
            st.dataframe(metrics_df, use_container_width=True, hide_index=True)

        matrix = irr_ner.get("confusion_matrix", [])
        if matrix:
            labels_for_matrix = LABELS + ["NO_ENTITY"]
            matrix_df = pd.DataFrame(matrix, index=labels_for_matrix[:len(matrix)],
                                     columns=labels_for_matrix[:len(matrix[0])])
            with st.expander("Confusion matrix NER"):
                st.dataframe(matrix_df, use_container_width=True)
        else:
            st.warning("File Kelp1_irr_ner.json tidak ditemukan.")

def main():
    st.set_page_config(page_title="Kelp1 Dataset Explorer", layout="wide")
    st.sidebar.title("Kelp1 Explorer")
    reviews_df, labels_df, entities_df, annotation_rows = load_data()

    if reviews_df.empty:
        st.error("Dataset tidak ditemukan atau kosong. Pastikan folder dataset berada satu level di atas file Streamlit.")
        st.stop()

    categories = sorted(reviews_df["category"].dropna().unique().tolist())
    selected_categories = st.sidebar.multiselect("Filter kategori usaha", categories, default=categories)
    selected_labels = st.sidebar.multiselect("Filter aspek-sentimen", LABELS)
    query = st.sidebar.text_input("Cari review / business_name")
    page = st.sidebar.radio("Halaman", ["Overview", "Browse Reviews", "Panel ABSA", "Statistics", "IRR"])

    filtered_reviews = filter_reviews(reviews_df, labels_df, selected_categories, selected_labels, query)

    st.sidebar.markdown("---")
    st.sidebar.write("Data terfilter:", len(filtered_reviews))
    st.sidebar.write("Total review:", len(reviews_df))

    if page == "Overview":
        page_overview(reviews_df, labels_df, entities_df, annotation_rows)
    elif page == "Browse Reviews":
        page_browse(reviews_df, labels_df, filtered_reviews)
    elif page == "Panel ABSA":
        page_absa(labels_df, entities_df, filtered_reviews, selected_labels)
    elif page == "Statistics":
        page_statistics(reviews_df, labels_df, entities_df, filtered_reviews)
    elif page == "IRR":

```

```
page_irr()
```

```
if __name__ == "__main__":  
    main()
```